

# Questions

**EVERY DAY 1 HOUR IN PYTHON- ALSO REVISION AND ACTIVE RECALL IS NEEDED FOR STRONG MEMORY**

=====

## 1. Difference Between List, Tuple, Set, Dictionary

| Type  | Ordered | Mutable | Duplicate |
|-------|---------|---------|-----------|
| List  | ✓       | ✓       | ✓         |
| Tuple | ✓       | ✗       | ✓         |
| Set   | ✗       | ✓       | ✗         |
| Dict  | ✓       | ✓       | Keys ✗    |

## CLASS and OBJECT:

class string:

```
def __init__(self,s):
```

```
    self.s=s
```

```
def rev_string(self):
```

```
    rev_str=""
```

```
    for i in self.s:
```

```
        rev_str= i+ rev_str
```

```
    return rev_str
```

```
def str_list(self):
```

```
    slist=self.s.split()
```

```
    return slist
```

```
def unique_ch(self):
```

```
    freq={}
```

```
    for i in self.s:
```

```

    if i in freq:
        freq[i]=freq[i] + 1
    else:
        freq[i]=1
return freq

```

```

a=string(input("Enter a String"))

```

```

print(a.rev_string())
print(a.str_list())
print(a.unique_ch())

```

**#you have string , and you need to find the largest word in string using python**

```

s="Mantu Kumar Dekka is good in Data Engineering"
print(f"Here is the String: {s}")
new_s=s.split(" ")
# print(new_s)

```

```

max=new_s[0]
for i in new_s:
    if len(i) >len(max):
        max=i

```

```

print(f"S0 Maximum length is :{max}")

```

**How to find the largest and smallest number from the integer array python?**

```

arr = [10, 3, 45, 7, 2]

```

```

max_val = max(arr)
min_val = min(arr)

```

—Or

```

arr = [10, 3, 45, 7, 2]

min_val = max_val = arr[0]

for num in arr:
    if num > max_val:
        max_val = num
    if num < min_val:
        min_val = num

print(max_val,min_val)

```

### **Get Unique record from a list?**

```

from collections import Counter

arr: list[int] = [2, 3, 5, 4, 5, 3, 4]

count = Counter(arr)

#print(count)

for i in count:
    if count[i] == 1:
        print(i)

```

### **write a function that returns a list containing all the mismatched words (case #sensitive) between two given input strings**

```

# For example:
string1= "Firstly this is the first string"
string2 = "Next is the second string"
# - output : ['Firstly', 'this', 'first', 'Next', 'second']
output=[]
string1=string1.split(" ")
string2=string2.split(" ")
for i in string1:
    if i not in string2:
        output.append(i)

```

```

for j in string2:
    if j not in string1:
        output.append(j)
#print(output)

```

**Given an array of integers, we would like to determine whether the array is monotonic (non-decreasing/non-increasing) or not ?**

#Examples:

```

## 1 2 5 5 8 // true // 9 4 4 2 2 // true // 1 4 6 3 // false
arr=[9,4,4,2,2]
flag=True
for i in range(len(arr)-1):
    if arr[i]>arr[i+1]:
        flag=False
        break
    else:
        flag=True
print(flag)

```

***# Write a python code in a list of common pair like 1+4= 5, 2+3=5***

```
lst = [2,1,4,3,5]
```

```
target=5
```

```

def add_sum(target,lst):
    hashmap={}
    for num,i in enumerate(lst):
        if target -num in hashmap:
            return [( target -num) ,num]
        else:
            hashmap[num]=i
print(add_sum(target,lst))

```

***## BUT I NEED PAIR?***

```
lst = [2, 1, 4, 3, 5]
```

```
target = 5
```

```
def add_all_pairs(target, lst):
```

```
    seen = set()
```

```
    pairs = []
```

```
    for num in lst:
```

```
        if target - num in seen:
```

```
            pairs.append((target - num, num))
```

```
            seen.add(num)
```

```
    return pairs
```

```
print(add_all_pairs(target, lst))
```

### **what is list comprehension?**

**List comprehension** in Python is a **compact way to create lists in one line** instead of using loops.

[**expression** for item in iterable if condition]

Normal way:

```
nums = [1,2,3,4,5]
```

```
result = []
```

```
for i in nums:
```

```
    result.append(i*i)
```

```
print(result)
```

### **List comprehension:**

```
nums = [1,2,3,4,5]
```

```
result = [i*i for i in nums]
```

```
print(result)
```

### Example 2: With condition:

```
nums = [1,2,3,4,5,6]
even = [i for i in nums if i % 2 == 0]
print(even)
```

### Example 3: With if-else:

```
nums = [1,2,3,4]
result = ["Even" if i%2==0 else "Odd" for i in nums]
print(result)
```

=====

### [expression for item in iterable if condition]

```
evens = [x for x in range(10) if x % 2 == 0]
print(evens) # [0, 2, 4, 6, 8]
```

—Flater a list:

```
matrix = [[1, 2], [3, 4], [5, 6]]
```

```
flat=[ j for i in matrix for j in i ]
```

```
print(flat)
```

=====

### **1 Reverse a string without using built-in functions**

```
def rev_string(s):
    new_str=""
    for i in s:
        new_str=i+new_str
    return new_str
m=input("Enter the name")
print(rev_string(m))
```

## **2 Find the first non-repeating character in a string**

```
def non_rep(n):
    from collections import Counter
    feq=Counter(n)
    print(feq)
    for i in n:
        if feq[i]==1:
            return i
m=input("Enter A String")
print(non_rep(m))
```

## **3 Count the frequency of elements in a list**

```
# Count the frequency of elements in a list
nums = [1,2,2,3,3,3]
```

```
freq = {}
```

```
for i in nums:
```

```
    if i in freq:
        freq[i] =freq[i] +1
```

```
    else:
        freq[i] = 1
```

```
print(freq)
```

```
# Or using Counter
```

```
from collections import Counter
```

```
feq=Counter(nums)
print(feq)
```

## **4 Find the top N most frequent elements**

```
#$ Find the top N most frequent elements
```

```
n=2
top3=feq.most_common(n)
print(top3)
```

#### **5 Remove duplicate values from a list without using set().**

```
l=[1,2,2,33,33,4,5,6,77,77,8,9,99,99,10]
print(l)
print(set(l))
```

```
res=[]
for i in l:
    if i not in res:
        res.append(i)
print(res)
```

#### **Remove duplicate values from a String without using builtin:**

```
s = "programming"
```

```
result = ""
```

```
for ch in s:
```

```
    if ch not in result:
        result += ch
```

```
print(result)
```

#### **Remove Duplicate Values from a Dictionary Without Built-in Function**

```
data = {
    'a': 100,
    'b': 200,
    'c': 100,
    'd': 300
}
```

```

}

result = {}

for key, value in data.items():

    if value not in result.values():
        result[key] = value

print(result)

```

=====

#### **6 Find the missing number in a given range**

```

numbers = [1, 2, 3, 5]

n = 5

missing = sum(range(1, n + 1)) - sum(numbers)

print(missing)

```

#### **7 Check whether two strings are anagrams**

**anagrams** :Both strings contain same characters with same frequency

```

str1 = "listen"
str2 = "silent"

if sorted(str1) == sorted(str2):
    print("Anagram")
else:
    print("Not Anagram")

```

#### **8 Merge two dictionaries and sum values of common keys**

```
dict1 = {'a': 100, 'b': 200, 'c': 300}
dict2 = {'b': 100, 'c': 200, 'd': 400}
```

```
result = {}
```

```
for key in dict1:
    result[key] = dict1[key]
```

```
for key in dict2:
```

```
    if key in result:
        result[key] += dict2[key]
    else:
        result[key] = dict2[key]
```

```
print(result)
```

### **9 Identify duplicate elements in a list**

```
numbers = [1, 2, 3, 2, 4, 5, 1, 6]
```

```
duplicates = []
```

```
for num in numbers:
```

```
    if numbers.count(num) > 1 and num not in duplicates:
        duplicates.append(num)
```

```
print(duplicates)
```

### **10.Remove the duplicate from a list:**

```
numbers = [1, 2, 3, 2, 4, 5, 1, 6]
```

```
duplicates = []
```

```
for num in numbers:
```

```
if num not in duplicates:
    duplicates.append(num)

print(duplicates)
```

### **10 Read a large file efficiently\_(memory-optimized approach)**

with open("large\_file.txt", "r") as file:

```
    for line in file:
        print(line.strip())
```

Why This is Memory Optimized: Reads one line at a time, instead of loading entire file into memory.

### **Bad Approach:**

```
file = open("large_file.txt", "r")
```

```
data = file.readlines()
```

Loads complete file into RAM, Very dangerous for GB/TB files.

### **11) Get non repeating character from the list:**

```
word="data engineering"
```

```
print(word)
```

```
from collections import Counter
```

```
word = "data engineering"
```

```
freq = Counter(word)
```

```
print(freq)
```

```
for ch in word:
```

```
    if ch != " " and freq[ch] == 1:
```

```
        print(ch)
```

```
        break
```

## **12. Find second largest element in a list using Python without builtin function**

```
numbers = [10, 5, 8, 20, 15]
```

```
largest = numbers[0]
```

```
second = numbers[0]
```

```
for num in numbers:
```

```
    if num > largest:
```

```
        second = largest
```

```
        largest = num
```

```
    elif num > second and num != largest:
```

```
        second = num
```

```
print(second)
```

## **#Find second largest element in a list using Python without builtin function**

```
lst = [10, 20, 4, 45, 99, 292]
```

```
largest = 0
```

```
second = 0
```

```
for i in lst:
```

```
    if i > largest:
```

```
        second = largest
```

```

largest = i

elif i > second and i != largest:
    second = i

print("Second Largest =", second)

```

=====

### ***Get First Pos Missing number***

```

lst = [1, 3, -1, 4]
#output=2
new_set = set(lst)
checkItem = []

for i in range(1, len(lst) + 2):
    if i not in new_set:
        checkItem.append(i)
        break

print(checkItem[0])

```

=====

### **# Get list of all missing numbers list**

```

lst = [1,2,3,5,8,10]
row_list = set(lst)
missing_list = []

for i in range(min(lst), max(lst) + 1):

    if i not in row_list:
        missing_list.append(i)

```

```

print(missing_list)
#o/p- [4, 6, 7, 9]

```

# # =====

### **# Get list of missing list + all**

```

list=[1,3,-1,4]

```

```
row=set(list)
check=[]

for i in range(min(list),max(list)+1):
    if i not in check:
        check.append(i)
print(check)
# o/p= [-1, 0, 1, 2, 3, 4]
```

### **# Count the missing number count**

```
lst = [1,3,-1,4,6]
row = set(lst)
count = 0
for i in range(min(lst), max(lst)+1):

    if i not in row:
        count += 1
print(count)
# o/p=3
```

### **#First Missing Positive (VERY FAMOUS)**

```
nums = [3,4,-1,1]
s = set(nums)
for i in range(1, max(nums)+2):

    if i not in s:
        print(i)
        break
# o/p =2
```

### **# Given a list of numbers, sort them based on their absolute difference from 10 using Python.**

```
def myfunc(n):
    return abs(10-n)

a = (5, 3, 1, 11, 2, 12, 17)
x = sorted(a, key=myfunc)
```

```
print(x)
```

### ***## Extend vs append***

```
def func(a, b):  
    a.extend(b) # o/p [4, 5, 4, 5, 6]  
    #a.append(b) -- o/p [4, 5, [4, 5, 6]]  
a = [4,5]  
b = [4,5,6]  
print(a)  
func(a,b)  
print(a)
```

### ***## find the largest difference between any two numbers in this list***

```
my_list = [1, 7, 9, 0, 3, 8]
```

```
print(my_list)
```

```
max=min=my_list[0]  
for i in my_list:  
    if i>max:  
        max=i  
    if i<min:  
        min=i  
print(max,min)  
print(f"Here is the Diff {max}-{min}")
```

### **or- using functions**

```
def max_min(a):  
    max_val = min_val = a[0]  
  
    for i in a:  
        if i > max_val:  
            max_val = i  
        if i < min_val:  
            min_val = i  
  
    print(max_val, min_val)  
    print(f"Here is the Diff {max_val - min_val}")
```

```
my_list = [1, 7, 9, 0, 3, 8]
max_min(my_list)
```

**You have a list of 1 million dictionaries in python need to filter.  
Only record amount is greater than 10,000.How to convert them  
to data frame**

```
import pandas as pd
```

```
# suppose data = list of 1M dictionaries
```

```
data = [
    {"name": "A", "salary": 5000},
    {"name": "B", "salary": 15000},
    {"name": "C", "salary": 8000}
]
```

```
high_salary = [d for d in data if d["salary"] > 7000]
# [{'name': 'B', 'salary': 15000}]
```

```
print(high_salary)
```

```
df=pd.DataFrame(high_salary)
print(df)
```

```
====using pyspark=====
```

```
df = spark.createDataFrame(data)
# Filter salary > 7000
high_salary_df = df.filter(col("salary") > 7000)
```

```
# Show result
high_salary_df.show()
```

**Write a function of adding two numbers-**

```
def add_two_number(a, b):
    return a + b

b = int(input("Enter a value of b: "))
c = int(input("Enter value of c: "))

res = add_two_number(b, c)

print(res)
```

=====

Deloitte-

```
logs = [
    "2026-04-01 10:00:00 INFO Service started",
    "2026-04-01 10:05:00 ERROR Database timeout",
    "2026-04-01 10:06:00 INFO Retry started",
    "2026-04-01 10:07:00 WARNING High memory usage",
    "2026-04-01 10:08:00 ERROR Connection lost"
]

result = {}

for log in logs:
    parts = log.split()

    timestamp = parts[0] + " " + parts[1]
    level = parts[2]

    if level not in result:
        result[level] = {"count": 0, "latest": timestamp}

    result[level]["count"] += 1

    # update latest timestamp
    if timestamp > result[level]["latest"]:
        result[level]["latest"] = timestamp
```

```
print(result)
```

```
=====
```

```
# Input (similar to your Scala structure) (AA)
```

```
l1 = [1, 2, [3, [4, 5, 6], 5, 6, 7]] # OUTPUT >> [1, 2, 3, 4, 5, 6, 7]
```

```
# Function to flatten list
```

```
def flatten(lst):
```

```
    result = []
```

```
    for item in lst:
```

```
        if isinstance(item, list):
```

```
            result.extend(flatten(item)) # recursive call
```

```
        else:
```

```
            result.append(item)
```

```
    return result
```

```
# Get flattened result
```

```
result = flatten(l1)
```

```
# Optional: remove duplicates and sort (as per your expected output)
```

```
result = sorted(set(result))
```

```
print(result)
```

```
=====
```

```
—Flater a list:
```

```
matrix = [[1, 2], [3, 4], [5, 6]]
```

```
flat=[ j for i in matrix for j in i ]
```

```
print(flat)
```

```
=====
```

input : sahil sahuo **(A) Python**

output: SaHiL sAhOo

```
text = "sahil sahuo"
```

```
result = []
```

```
for word in text.split():
    new_word = ""
    for i, ch in enumerate(word):
        if i % 2 == 0:
            new_word += ch.upper()
        else:
            new_word += ch.lower()
    result.append(new_word)
```

```
final_output = " ".join(result)
print(final_output)
```

```
=====
```

In Python(not pyspark): **(AAA)**

```
=====
```

Input:

```
=====
```

```
list1= ["ind", "aus", "eng", "nz"]
```

```
=====
```

Output:

```
=====
```

India vs Australia

India vs England

India vs New Zealand

Australia vs England

Australia vs New Zealand

England vs New Zealand

**sol:**

```
list1 = ["ind", "aus", "eng", "nz"]
```

```

mapping = {
    "ind": "India",
    "aus": "Australia",
    "eng": "England",
    "nz": "New Zealand"
}

for i in range(len(list1)):
    for j in range(i + 1, len(list1)):
        team1 = mapping[list1[i]]
        team2 = mapping[list1[j]]
        print(f"{team1} vs {team2}")

```

=====

Question from IBM: Python (AAA)

*# Input:*

```
data=[(("a","aa",1),("b","bb",2),("c","cc",3),("d","dd",4))]
```

*# Output:*

```
# List(("a",1),("b",4),("c",9),("d",16))
```

*list → tuple → tuple*

**So you must iterate twice:**

1. **outer → the big tuple**
2. **inner → each record like ("a","aa",1)**

```
data = [(("a","aa",1),("b","bb",2),("c","cc",3),("d","dd",4))]
```

```
result = []
```

```

for group in data:      # outer loop
    for item in group:  # inner loop
        result.append((item[0], item[2]**2))

```

```
print(result)
```

=====

```
data = [("a","aa",1),("b","bb",2),("c","cc",3),("d","dd",4)]
```

```
result = []
```

```
for item in data:
```

```
    result.append((item[0], item[2]**2))
```

```
print(result)
```

### **Reverse string without using built-in functions:**

```
def reverse_string(s):
```

```
    result = ""
```

```
    for char in s:
```

```
        result = char + result
```

```
    return result
```

```
print(reverse_string("Cognizant")) # Output: tnizgnoC
```

### **Write a function without using builtin function for SWAP CASE- swapcase()**

```
def swap_case(s):
```

```
    result = ""
```

```
    for c in s:
```

```
        if c.islower():
```

```
            result =result+c.upper()
```

```
        elif c.isupper():
```

```
            result =result+c.lower()
```

```
        else:
```

```
            result =result+c
```

```
    return result
```

```
print(swap_case(input("Enter a string: ")))
```

### **Count Vowel and Consonant in a given word:**

```
vowel = ['a', 'e', 'i', 'o', 'u']

word = input("Enter a word: ")
count = 0
cct=0

for letter in word:
    if letter not in vowel:
        cct += 1
    else:
        count += 1
        print("Vowel", count)

print("consonant count",cct)
```

### **function:**

```
def vowel_consonant(string):
    vowels = ['a', 'e', 'i', 'o', 'u']
    vcount = 0
    ccount = 0
    for char in string.lower():
        if char in vowels:
            vcount += 1
        else:
            ccount += 1

    return vcount, ccount

print(vowel_consonant(input("Enter A String:")))
```

### **Counting the number of occurrences of a character in a **String**:**

```
def char_count(string):
    count = 0
    char="a"
```

```

for i in string:
    if char == i:
        count =count+1
return count
print(char_count(input("Enter a string: ")))

```

### **Counting the number of occurrences of a character in a List:**

```
lst = ["apple", "banana", "avocado"] OR
```

```
lst = ['a', 'b', 'a', 'c', 'a']
```

```

from collections import Counter
print(Counter(lst))

```

### **Counting the number of occurrences of a character in a LIST>>Dictanry: IBM**

a = [1,2,3,2,1,4,1], Write a code to convert this list into dictionary having the key as unique number and values of that key should be the count of that number in list like a = {1:3, 2:2, 3:1, 4:1}

```
a = [1, 2, 3, 2, 1, 4, 1]
```

```
freq = {}
```

```

for num in a:
    if num in freq:
        freq[num] += 1
    else:
        freq[num] = 1

```

```
print(freq)
```

### **## Remove duplicate from a String: (AA).**

```
s = "programming"
```

```

result = ""

for ch in s:
    if ch not in result: # check in result instead of set
        result += ch

print(result)

```

**## Remove duplicate from a list: (AAA).**

```
A=[1,2,2,3,3,3,3,4,5,6,7,8]
```

```

res=[]
for i in A:
    if i not in res:
        res.append(i)
print(res)

```

**Counting the number of occurrences of a character in a**  
**>Dictanry:**

=====

**#Writing Fibonacci series-**

```

a,b=1,2
series=[]
n=int(input("Please input the number of elements: "))

for i in range(n):
    series.append(a)
    a,b=b,a+b
print(series)

```

—————SPLIT and JOIN—————

```
def split_and_join(line):
    a = line.split(" ")
    b = "-" .join(a)
    return b

if __name__ == '__main__':
    line = input("Enter a line: ")
    result = split_and_join(line)
    print(result)
```

## ————— GIVING SPACE ON BASIS ON CAP LETTER —————

(A)

```
def Separate_Words(text):
    newstring = ""
    for char in text:
        if char.isupper():
            newstring = newstring + " " + char
        else:
            newstring = newstring + char
    return newstring
```

# Example usage

```
print(Separate_Words(input("Enter a sentence in Small and Cap  
Letter: Without Space")))
```

## ————— INSERT INTO STRING —————

```
def mutate_string(string, position, character):
    l = list(string)
    l[position] = character
    string = ''.join(l)
    return string
```

```
if __name__ == '__main__':
    s = input("Enter a string: ")
```

```
i, c = input("Enter Pos and Char").split()
s_new = mutate_string(s, int(i), c)
print(s_new)
```

```
=====
===
```

### ——Count specific words from a file——

```
with open("/Users/mantukumardeka/Desktop/delete.txt", 'r') as file:
    data1 = file.read()
    data=data1.lower()

    print("Here the file:", data)

    data_split = data.split()

    print("After split:", data_split)

from collections import Counter

final_Data = Counter(data_split)
print(final_Data)
print("Count of Night:", final_Data["night"])
```

```
=====
=====
```

**Write a python code to find the Ids that contains duplicate key values in a list of dictionaries. (A)**

```
sample = [{"id": 1, "Name": "ABC"},
           {"id": 2, "Name": "XYZ"},
           {"id": 3, "Name": "ABC"},
           {"id": 4, "Name": "PQR"},
           {"id": 5, "Name": "XYZ"}]
```

```
sample = [  
    {"id": 1, "Name": "ABC"},  
    {"id": 2, "Name": "XYZ"},  
    {"id": 3, "Name": "ABC"},  
    {"id": 4, "Name": "PQR"},  
    {"id": 5, "Name": "XYZ"}  
]
```

*# Step 1: Count occurrences of each Name*

```
name_count = {}  
for item in sample:  
    name = item["Name"]  
    name_count[name] = name_count.get(name, 0) + 1
```

*# Step 2: Find IDs where Name occurs more than once*

```
duplicate_ids = [item["id"] for item in sample if  
name_count[item["Name"]] > 1]
```

```
print("IDs with duplicate Names:", duplicate_ids)
```

===== (A)

```
sentences = [  
  
    "Python is great for data development!",  
  
    "Data developers love Python and Pandas.",  
  
    "Python is powerful, flexible, and easy to learn.",  
  
    "Data engineering and Python go hand in hand."  
  
]
```

#1. Normalize the text (lowercase, remove punctuation).

#2. Count the frequency of each word.

#3. Find the top N most frequent words.

#4. Return results as a sorted list.

```
import re
text = " "×join(sentences)×lower()
clean_text = re.sub(r"^\w\s]", "", text)

# Step 2: Split into words
words = clean_text.split()

from collections import Counter

# Step 3: Count frequency
word_counts = Counter(words)

# Step 4: Get top N frequent words
N = 5
top_n = word_counts.most_common(N)

print(top_n)

# Step 4: Return sorted list
result = sorted(top_words, key=lambda x: (-x[1], x[0]))
```

=====

**# What is the output? (A)**

a="aaabbcccdccc" → a3b2c6d3

*# InPUT : aaabbca, output : 3a2b1c1a . Write python code.*

```
a = "aaabbca"
```

```
res = ""
count = 1
```

```

for i in range(len(a)):
    if i+1 < len(a) and a[i] == a[i+1]:
        count += 1
    else:
        res += str(count) + a[i]    # This will change
        res += a[i] +
str(count)
        count = 1

print(res)

```

=====

**#Given an integer n find the sum of the first n natural number.**

**Input: n = 10 Output: 55 Explanation: 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 = 55.**

```

def nSum(n):
    # Using the mathematical formula for sum of first n natural
    numbers
    ans = n × (n + 1) // 2
    return ans

```

```

n = int(input("Enter N: "))
print(nSum(n))

```

OR-

**Alternative Solution — Using Loop (For Beginners)**

```

def nSum(n):
    ans = 0
    for i in range(1, n + 1):
        ans += i
    return ans

```

=====

### **# Program to compute LCM and GCD of two numbers**

```
import math
```

```
def lcm_and_gcd(a, b):  
    gcd = math.gcd(a, b)      # Calculate GCD  
    lcm = (a × b) // gcd      # Calculate LCM using formula  
    return [lcm, gcd]
```

```
# Input from the user  
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))
```

```
# Compute LCM and GCD  
result = lcm_and_gcd(a, b)  
print(f"LCM: {result[0]}, GCD: {result[1]}")
```

### **# without bulidin\_function:**

```
def compute_gcd(a, b):  
    while b != 0:  
        a, b = b, a % b  
    return a
```

```
def compute_lcm(a, b):  
    gcd = compute_gcd(a, b)  
    lcm = (a × b) // gcd  
    return lcm
```

```
# Input  
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))
```

```
gcd = compute_gcd(a, b)  
lcm = compute_lcm(a, b)
```

```
print(f"GCD: {gcd}, LCM: {lcm}")
```

```
=====
==
```

### **# Write a Program to convert Integer to Binary number:**

```
class Solution:
    def decToBinary(self, n):
        # Edge case: if number is 0
        if n == 0:
            return "0"

        binary = "" # To store binary digits

        # Repeatedly divide by 2 and collect remainders
        while n > 0:
            remainder = n % 2
            binary = str(remainder) + binary # Add remainder to the left
            side
            n = n // 2 # Integer division

        return binary
```

```
# ----- MAIN PROGRAM -----
# Take user input
n = int(input("Enter a decimal number: "))
```

```
# Create object of Solution class
sol = Solution()
```

```
# Call the function and print result
print("Binary representation:", sol.decToBinary(n))
```

```
=====
```

```

class Solution:
    def decToBinary(self, n):
        # code here

        if n == 0:
            return "0"

        binary = "" # To store binary digits

        # Repeatedly divide by 2 and collect remainders
        while n > 0:
            remainder = n % 2
            binary = str(remainder) + binary # Add remainder to the left
            n = n // 2 # Integer division

        return binary
=====

```

### **# Print Largest number:**

```

a = 1
b = 2
c = 3

if a>b and a>c:
    print(a)
elif b>a and b>c:
    print(b)
elif c>a and c>b:
    print(c)

```

```

def greatest_of_three(a, b, c):
    if a > b and a > c:
        return a
    elif b > a and b > c:
        return b

```

```
else:  
    return c
```

\_\_\_\_\_

```
def greatest_of_three(a, b, c):  
    print(max(a, b, c))
```

### **#How to find the largest and smallest number from the integer array.python?**

```
arr = [12, 45, 7, 89, 34, 2]  
  
smallest = largest = arr[0]  
  
for num in arr:  
    if num < smallest:  
        smallest = num  
    elif num > largest:  
        largest = num  
  
print("Smallest:", smallest)  
print("Largest:", largest)
```

—

```
arr = [12, 45, 7, 89, 34, 2]  
  
smallest = min(arr)  
largest = max(arr)  
  
print("Smallest:", smallest)  
print("Largest:", largest)
```

—

```
arr = [12, 45, 7, 89, 34, 2]
```

```
arr.sort()
smallest = arr[0]
largest = arr[-1]

print("Smallest:", smallest)
print("Largest:", largest)
```

\_\_\_\_\_

***find the unique number present in the given array in python?***  
The **distinct elements** (remove duplicates)

**=====SUM of TWO NUMBERS VVI=====**

**( A )**

**Solution:**

```
nums = [2, 7, 11, 15]
target = 9

def twosum(nums, target):
    hashmap = {}

    for i, num in enumerate(nums):
        if target - num in hashmap:
            return [hashmap[target - num], i]
        hashmap[num] = i

    return []

print(twosum(nums, target))
```

\_\_\_\_\_or\_\_\_\_\_

```
nums=[1,5,2,4,7,6]
target=11
```

```
def addtwo(nums,target):
    hashmap={}

    for i,num in enumerate(nums):
        if target -num in hashmap:
            return [hashmap[target -num],i]
        hashmap[num]=i
    print(addtwo(nums,target))
```

=====

(A)

### **# LIST ITERATION:**

***Sort a LST using built in and without Built in functions:***

- ✓ Bubble Sort
- ✓ Selection Sort
- ✓ Insertion Sort
- ✓ Merge Sort
- ✓ Quick Sort
- ✓ Heap Sort

All implemented **without using sorted() or list.sort()**.

***--Using sorted() and sort() method***

```
A=[11,223,4,44,55,234,445,67,67]
print(sorted(A))
```

```
A=[11,223,4,44,55,234,445,67,67]
A.sort()
print(A)
```

✓ Bubble Sort

```
A=[11,223,4,44,55,234,445,67,67]
```

```
for i in range(len(A)):
    for j in range(0, len(A)-i-1):
        if A[j] > A[j+1]:
            A[j], A[j+1] = A[j+1], A[j]

print(A)
```

### ✓ Selection Sort. (A)

```
A=[11,223,4,44,55,234,445,67,67]
```

```
for i in range(len(A)):
    min_index = i
    for j in range(i+1, len(A)):
        if A[j] < A[min_index]:
            min_index = j
    A[i], A[min_index] = A[min_index], A[i]

print(A)
```

### # Remove duplicate from list: (A)

```
A=[1,2,2,3,3,3,3,4,5,6,7,8]
```

```
A1=set(a)
print(f"Removing duplicate by using set:{A1}")
```

```
print("By using Loop: removing duplicates")
```

```
A=[1,2,2,3,3,3,3,4,5,6,7,8]
```

```
lst=[]
```

```
for i in A:
    if i not in lst:
        lst.append(i)
print(lst)
```

=====

### # Write a python program to count the number of words in a sentence:

Using **len(), split() strip():**

```
sentence = "Python is very easy to learn"
print(f"Before using anytung:{sentence}")
words = sentence.split()
```

```
#print(words)
print(f"After using SPLIT : {words}")
count = len(words)
```

```
print(f"Number of words:", {count})
-----
```

### **Without Using Built-in split() — Interview Version: (A)**

```
words= " Mantu Kumar Deks  is very expert  in Python programming
"
```

```
count = 0
in_words = False
```

```
for ch in words:
    if ch != ' ' and not in_words:
        count = count + 1
        in_words = True
    elif ch == ' ' and in_words:
        in_words = False
```

```
print(count)
```

=====

### ### Python Questions-

write code using python s1 and s2 input s is output?  
input

===

s1 = "Hi, My name is P"

s2 = "My name is K"

### output

s = "Hi, P K"

```

s1 = "Hi, My name is P"

s2 = "My name is K"

# Remove common part from s2

common = "My name is"

s2\_unique = s2.replace(common, "").strip()

# Extract initial part from s1 before common

s1\_prefix = s1.split(common)[0].strip()

# Extract last char from s1's unique part (optional based on example)

s1\_last = s1.split()[-1]

# Combine

s = f"{s1\_prefix} {s1\_last} {s2\_unique}".strip()

print("s =", s)

```

=====

1)Write a program to Find Prime numbers using python?

***# Program to find prime numbers in a given range***

```

def is_prime(n):
    """Check if a number is prime"""
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

# Example: Find prime numbers between 1 and 50
prime_numbers = [num for num in range(1, 51) if is_prime(num)]

print("Prime numbers between 1 and 50:", prime_numbers)

```

---

Scenario question, Count the odd one out numbers?

---

```

list_data = [
    ['11', 'manish', '10008', 'US', '01/05/2022', 'Y'],
    ['12', 'raj', '28888', 'India', '02/05/2022', 'Y'],
    ['13', 'meg', '15000', 'India', '03/05/2022', 'Y']
]

india = []
us = []

for i in list_data:
    if i[3] == 'India':    # country is at index 3
        india.append(int(i[2])) # salary is at index 2
    else:
        us.append(int(i[2]))

print("India salaries:", india)
print("US salaries:", us)

# Calculate sums
sum_india = sum(india)
sum_us = sum(us)

```

```
print(f"Total India salary spend: {sum_india}, US salary spend: {sum_us}")
```

=====

**Exercise 2: Print characters from a string that are present at an even index number?**

```
string=input("enter the text: ")
x=list(string)
print(string)
for i in x[0::2]:
    print(i,end=" ")
```

=====

**Exercise 4B: Write a program to create a new string made of the middle three characters of an input string.**

ex.JaSonAy-->Son  
JhonDipPeta -->Dip

```
name = "JaSonAy"
l = len(name)

if l % 2 == 0:
    print("not possible")
else:
    c = l // 2
    x = name[c - 1]
    y = name[c]
    z = name[c + 1]
    result = "".join([x, y, z])
    print(result)
```

=====

# Python Interview Questions & Answers

## 1. Basic Python Concepts

**Question: What are Python data types?**

**Answer:** Python has several built-in data types:

- **Numeric:** int, float, complex
- **Sequence:** str, list, tuple
- **Mapping:** dict
- **Set:** set, frozenset
- **Boolean:** bool
- **Binary:** bytes, bytearray, memoryview

**Question: What is the difference between list and tuple?**

**Answer:**

| Feature     | List                | Tuple                  |
|-------------|---------------------|------------------------|
| Syntax      | [1, 2, 3]           | (1, 2, 3)              |
| Mutable     | Yes (can change)    | No (cannot change)     |
| Ordered     | Yes                 | Yes                    |
| Methods     | Many (append, etc.) | Fewer (count, index)   |
| Performance | Slightly slower     | Faster (read-only use) |
| Use case    | Dynamic data        | Fixed, constant data   |

**Question: What is the difference between append and extend?**

**Answer:**

```
L1 = [2, 3, 4, 5]
```

```
L2 = [6, 7, 8, 9]
```

```
# append() - adds entire L2 as single element
```

```
L1.append(L2)
```

```
print(L1) # Output: [2, 3, 4, 5, [6, 7, 8, 9]]
```

# extend() - adds each element of L2 individually

```
L1 = [2, 3, 4, 5]
```

```
L1.extend(L2)
```

```
print(L1) # Output: [2, 3, 4, 5, 6, 7, 8, 9]
```

**Question: What is mutable and immutable in Python?**

**Answer:**

- **Mutable:** Objects that can be changed after creation
  - Examples: list, dict, set
- **Immutable:** Objects that cannot be changed after creation
  - Examples: int, float, str, tuple

## 2. Python Data Structures

**Question: What are dictionaries in Python?**

**Answer:** Dictionaries are used to store key-value pairs.

# Creating a dictionary

```
my_dict = {1: 'HBO Max', 2: 'Netflix', 3: 'Prime Video'}
```

# Accessing values

```
print(my_dict[1]) # Output: HBO Max
```

# Adding new key-value pair

```
my_dict[4] = 'Disney+'
```

# Dictionary methods

```
keys = my_dict.keys()
```

```
values = my_dict.values()
```

```
items = my_dict.items()
```

**Question: What is the difference between List, Tuple, Set and Dictionary?**

**Answer:**

| Type  | Syntax  | Ordered | Mutable | Duplicates | Use Case        |
|-------|---------|---------|---------|------------|-----------------|
| List  | [1,2,3] | Yes     | Yes     | Yes        | Sequential data |
| Tuple | (1,2,3) | Yes     | No      | Yes        | Fixed data      |
| Set   | {1,2,3} | No      | Yes     | No         | Unique elements |

|      |         |               |     |              |                        |
|------|---------|---------------|-----|--------------|------------------------|
| Dict | {1:'a'} | Yes<br>(3.7+) | Yes | No<br>(keys) | Key-<br>value<br>pairs |
|------|---------|---------------|-----|--------------|------------------------|

**Question: Write Python code to add dictionary values**

**Input:** {"a": 100, "b": 200, "c": 300}

**Output:** 600

**Answer:**

```
input_dict = {"a": 100, "b": 200, "c": 300}
```

```
total_sum = sum(input_dict.values())
```

```
print(total_sum) # Output: 600
```

### 3. String Operations

**Question: Write Python code to reverse a string**

**Answer:**

```
# Method 1: Using slicing
```

```
s = "Hello World"
```

```
reversed_s = s[::-1]
```

```
print(reversed_s) # Output: dlroW olleH
```

```
# Method 2: Using reversed() function
```

```
s = "Hello World"
```

```
reversed_s = ''.join(reversed(s))
```

```
print(reversed_s)
```

```
# Method 3: Manual reversal
```

```
def reverse_string(s):
```

```
    result = ""
```

```
    for char in s:
```

```
        result = char + result
```

```
    return result
```

**Question: Reverse words in a sentence**

**Input:** "Spark is Fun"

**Output:** "Fun is Spark"

**Answer:**

```
def reverse_words(sentence):
```

```
    words = sentence.split()
```

```
    return ' '.join(reversed(words))
```

```
input_str = "Spark is Fun"
```

```
output = reverse_words(input_str)
```

```
print(output) # Output: Fun is Spark
```

**Question: Remove whitespaces from string**

**Answer:**

```
# Remove all spaces
text = " Hello World "
cleaned = text.replace(" ", "")
print(cleaned) # Output: HelloWorld
```

```
# Remove leading/trailing spaces
cleaned = text.strip()
print(cleaned) # Output: Hello World
```

```
# Remove all whitespace characters
import re
cleaned = re.sub(r'\s+', '', text)
print(cleaned) # Output: HelloWorld
```

## 4. List Operations

**Question: Write Python code to reverse a list**

**Answer:**

```
# Method 1: Using slicing
lst = [3, 2, 1, 0]
reversed_lst = lst[::-1]
print(reversed_lst) # Output: [0, 1, 2, 3]
```

```
# Method 2: Using reverse() method
lst = [3, 2, 1, 0]
lst.reverse()
print(lst) # Output: [0, 1, 2, 3]
```

```
# Method 3: Using reversed() function
lst = [3, 2, 1, 0]
reversed_lst = list(reversed(lst))
print(reversed_lst) # Output: [0, 1, 2, 3]
```

**Question: Find the largest and smallest number from an integer array**

**Answer:**

```
def find_min_max(arr):
    if not arr:
        return None, None

    minimum = min(arr)
    maximum = max(arr)
    return minimum, maximum
```

```
# Example usage
```

```
numbers = [5, 1, 50, 75, 60, 100]
min_val, max_val = find_min_max(numbers)
print(f"Minimum: {min_val}, Maximum: {max_val}")
# Output: Minimum: 1, Maximum: 100
```

**Question: Find unique numbers in an array**

**Answer:**

```
def find_unique_numbers(arr):
    return list(set(arr))
```

# Example

```
numbers = [1, 2, 3, 2, 4, 3, 5]
unique = find_unique_numbers(numbers)
print(unique) # Output: [1, 2, 3, 4, 5]
```

**Question: Remove duplicates from list**

**Answer:**

# Method 1: Using set

```
lst = [1, 2, 3, 4, 2, 3, 2]
unique_list = list(set(lst))
print(unique_list)
```

# Method 2: Preserving order

```
def remove_duplicates(lst):
    seen = set()
    result = []
    for item in lst:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result
```

```
lst = [1, 2, 3, 4, 2, 3, 2]
result = remove_duplicates(lst)
print(result) # Output: [1, 2, 3, 4]
```

**Question: Retrieve only duplicates from the list**

**Answer:**

```
def find_duplicates(lst):
    seen = set()
    duplicates = set()

    for item in lst:
        if item in seen:
            duplicates.add(item)
```

```
else:
    seen.add(item)
```

```
return list(duplicates)
```

```
lst = [1, 2, 3, 4, 2, 3, 2]
duplicates = find_duplicates(lst)
print(duplicates) # Output: [2, 3]
```

## 5. Mathematical Operations

**Question: Sum of elements in a list without using sum()**

**Answer:**

```
def calculate_sum(lst):
    total = 0
    for num in lst:
        total += num
    return total
```

```
numbers = [1, 2, 3, 4, 5]
result = calculate_sum(numbers)
print(result) # Output: 15
```

**Question: Check if array is monotonic (non-decreasing/non-increasing)**

**Examples:**

- [1, 2, 5, 5, 8] → True (non-decreasing)
- [9, 4, 4, 2, 2] → True (non-increasing)
- [1, 4, 6, 3] → False

**Answer:**

```
def is_monotonic(arr):
    increasing = all(arr[i] <= arr[i+1] for i in range(len(arr)-1))
    decreasing = all(arr[i] >= arr[i+1] for i in range(len(arr)-1))
    return increasing or decreasing
```

# Test cases

```
print(is_monotonic([1, 2, 5, 5, 8])) # True
print(is_monotonic([9, 4, 4, 2, 2])) # True
print(is_monotonic([1, 4, 6, 3]))    # False
```

**Question: Write a program to find prime numbers**

**Answer:**

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
```

```

        if n % i == 0:
            return False
    return True

```

```

def find_primes_up_to(n):
    primes = []
    for num in range(2, n + 1):
        if is_prime(num):
            primes.append(num)
    return primes

```

```

# Find primes up to 20
primes = find_primes_up_to(20)
print(primes) # Output: [2, 3, 5, 7, 11, 13, 17, 19]

```

**Question: Write a program for Fibonacci series**

**Answer:**

```

def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

```

```

def fibonacci_series(n):
    series = []
    for i in range(n):
        series.append(fibonacci(i))
    return series

```

```

# Generate first 10 Fibonacci numbers
fib_series = fibonacci_series(10)
print(fib_series) # Output: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

```

## 6. String Manipulation Problems

**Question: Count word frequency in a string**

**Answer:**

```

def word_count(text):
    words = text.lower().split()
    word_freq = {}

    for word in words:
        # Remove punctuation
        word = word.strip('.,!?:;"')
        if word in word_freq:
            word_freq[word] += 1

```

```
else:
    word_freq[word] = 1
```

```
return word_freq
```

```
text = "Hello world hello Python world"
result = word_count(text)
print(result) # Output: {'hello': 2, 'world': 2, 'python': 1}
```

**Question: Find mismatched words between two strings**

**Example:**

- String1: "Firstly this is the first string"
- String2: "Next is the second string"
- Output: ['Firstly', 'this', 'first', 'Next', 'second']

**Answer:**

```
def find_mismatched_words(string1, string2):
    words1 = string1.split()
    words2 = string2.split()
```

```
    mismatched = []
```

```
    # Words in string1 but not in string2
    for word in words1:
        if word not in words2:
            mismatched.append(word)
```

```
    # Words in string2 but not in string1
    for word in words2:
        if word not in words1:
            mismatched.append(word)
```

```
    return mismatched
```

```
string1 = "Firstly this is the first string"
string2 = "Next is the second string"
result = find_mismatched_words(string1, string2)
print(result) # Output: ['Firstly', 'this', 'first', 'Next', 'second']
```

## 7. Advanced Python Concepts

**Question: What are lambda functions? Give an example.**

**Answer:**

# Lambda function syntax: lambda arguments: expression

# Example 1: Square function

```
square = lambda x: x * x
print(square(5)) # Output: 25
```

```
# Example 2: Add two numbers
add = lambda a, b: a + b
print(add(3, 7)) # Output: 10
```

```
# Example 3: Using with map()
numbers = [1, 2, 3, 4]
squared = list(map(lambda x: x**2, numbers))
print(squared) # Output: [1, 4, 9, 16]
```

```
# Example 4: Using with filter()
numbers = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens) # Output: [2, 4, 6]
```

**Question: What is list comprehension?**

**Answer:** List comprehension provides a concise way to create lists.

# Syntax: [expression for item in iterable if condition]

```
# Example 1: Square numbers
squares = [x**2 for x in range(1, 6)]
print(squares) # Output: [1, 4, 9, 16, 25]
```

```
# Example 2: Even numbers
evens = [x for x in range(10) if x % 2 == 0]
print(evens) # Output: [0, 2, 4, 6, 8]
```

```
# Example 3: String manipulation
words = ["hello", "world", "python"]
upper_words = [word.upper() for word in words]
print(upper_words) # Output: ['HELLO', 'WORLD', 'PYTHON']
```

**Question: What are decorators in Python?**

**Answer:**

```
def my_decorator(func):
    def wrapper():
        print("Something before the function")
        func()
        print("Something after the function")
    return wrapper
```

```
@my_decorator
```

```
def say_hello():
    print("Hello!")

# Usage
say_hello()
# Output:
# Something before the function
# Hello!
# Something after the function

# Decorator with parameters
def repeat(times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator
```

```
@repeat(3)
def greet(name):
    print(f"Hello, {name}!")
```

`greet("Alice")` # Prints "Hello, Alice!" three times

**Question: What is multithreading in Python?**

**Answer:**

```
import threading
import time
```

```
def worker(name):
    for i in range(3):
        time.sleep(1)
        print(f"Worker {name}: {i}")
```

```
# Create threads
thread1 = threading.Thread(target=worker, args=("A",))
thread2 = threading.Thread(target=worker, args=("B",))
```

```
# Start threads
thread1.start()
thread2.start()
```

```
# Wait for threads to complete
thread1.join()
thread2.join()
```

```
print("All threads completed")
```

## **8. Exception Handling**

**Question: What is exception handling in Python?**

**Answer:**

```
# Basic try-catch structure
```

```
try:
```

```
    result = 10 / 0
```

```
except ZeroDivisionError:
```

```
    print("Cannot divide by zero!")
```

```
except Exception as e:
```

```
    print(f"An error occurred: {e}")
```

```
else:
```

```
    print("No exceptions occurred")
```

```
finally:
```

```
    print("This always executes")
```

```
# Multiple exception handling
```

```
def safe_divide(a, b):
```

```
    try:
```

```
        result = a / b
```

```
        return result
```

```
    except ZeroDivisionError:
```

```
        return "Error: Division by zero"
```

```
    except TypeError:
```

```
        return "Error: Invalid types"
```

```
    except Exception as e:
```

```
        return f"Unexpected error: {e}"
```

```
# Usage examples
```

```
print(safe_divide(10, 2)) # Output: 5.0
```

```
print(safe_divide(10, 0)) # Output: Error: Division by zero
```

```
print(safe_divide(10, "a")) # Output: Error: Invalid types
```

## **9. Date and Time Operations**

**Question: Print current time, date, and day in Python**

**Answer:**

```
from datetime import datetime
```

```
# Get current datetime
now = datetime.now()

# Print different formats
print("Current Date:", now.date())
print("Current Time:", now.strftime("%H:%M:%S"))
print("Day of Week:", now.strftime("%A"))
print("Full DateTime:", now.strftime("%Y-%m-%d %H:%M:%S"))

# Output example:
# Current Date: 2024-01-15
# Current Time: 14:30:45
# Day of Week: Monday
# Full DateTime: 2024-01-15 14:30:45
```

## 10. Object-Oriented Programming

**Question: Can we use classes in Python?**

**Answer:** Yes, Python supports object-oriented programming.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def get_info(self):
        return f"Employee: {self.name}, Salary: ${self.salary}"

    def give_raise(self, amount):
        self.salary += amount
        return f"New salary for {self.name}: ${self.salary}"
```

```
# Usage
emp1 = Employee("John", 50000)
print(emp1.get_info())
print(emp1.give_raise(5000))
```

**Question: Define a class with two methods - one to get input string and another to print odd indexed letters**

**Answer:**

```
class OddLetterPrinter:
    def get_input_string(self):
        self.input_string = input("Enter a string: ")

    def print_odd_letters(self):
```

```

    if not hasattr(self, 'input_string'):
        print("Error: input string not set")
        return

    odd_letters = ""
    for i in range(len(self.input_string)):
        if i % 2 == 0: # 0-based indexing, so even indices are "odd
positions"
            odd_letters += self.input_string[i]
    print(odd_letters)

# Usage
printer = OddLetterPrinter()
printer.get_input_string() # User inputs "Hello"
printer.print_odd_letters() # Output: "Hlo" (positions 0, 2, 4)

```

## 11. File Operations

### Question: Read text file in Python

#### Answer:

```

# Method 1: Basic file reading
def read_file(filename):
    try:
        with open(filename, 'r') as file:
            content = file.read()
            return content
    except FileNotFoundError:
        return "File not found"
    except Exception as e:
        return f"Error reading file: {e}"

# Method 2: Read line by line
def read_file_lines(filename):
    try:
        with open(filename, 'r') as file:
            lines = file.readlines()
            return [line.strip() for line in lines]
    except Exception as e:
        return f"Error: {e}"

# Method 3: Read and process large files
def process_large_file(filename):
    try:
        with open(filename, 'r') as file:

```

```

        for line in file:
            # Process each line individually
            print(line.strip())
    except Exception as e:
        print(f"Error: {e}")

```

## 12. Practical Programming Problems

**Question: Count frequency of characters in a string**

**Answer:**

```

def char_frequency(string):
    freq = {}
    for char in string:
        if char in freq:
            freq[char] += 1
        else:
            freq[char] = 1
    return freq

```

# Example

```

text = "hello world"
result = char_frequency(text)
print(result) # Output: {'h': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 1, 'w': 1, 'r': 1, 'd': 1}

```

**Question: Check if a string is palindrome**

**Answer:**

```

def is_palindrome(s):
    # Remove spaces and convert to lowercase
    cleaned = s.replace(" ", "").lower()
    return cleaned == cleaned[::-1]

```

# Test cases

```

print(is_palindrome("racecar")) # True
print(is_palindrome("A man a plan a canal Panama")) # True
# (ignoring spaces)
print(is_palindrome("hello")) # False

```

**Question: Find the second largest number in a list without using built-in functions**

**Answer:**

```

def second_largest(numbers):
    if len(numbers) < 2:
        return None

    first = second = float('-inf')

```

```

for num in numbers:
    if num > first:
        second = first
        first = num
    elif num > second and num != first:
        second = num

return second if second != float('-inf') else None

```

```

# Test
nums = [10, 4, 20, 99, 4, 51, 10, 20, 51]
result = second_largest(nums)
print(result) # Output: 51

```

## Common Python Interview Topics Summary:

1. **Basic Data Types and Operations**
2. **String Manipulations**
3. **List Operations and Algorithms**
4. **Dictionary and Set Operations**
5. **Object-Oriented Programming**
6. **Exception Handling**
7. **Lambda Functions and Functional Programming**
8. **File I/O Operations**
9. **Mathematical Algorithms** (Prime numbers, Fibonacci)
10. **Date/Time Operations**

These questions represent the most commonly asked Python concepts in data engineering interviews, focusing on practical problem-solving skills and fundamental programming concepts.

=====

Solve this-

$x = 10$

Output - {0:1,2:3,4:5,6:7,8:9,10:11}

It = {z:z+1 for z in range(0,x,2)}

---

```
x = '1222311'
```

```
[(1, 1), (3, 2), (1, 3), (2, 1)]
```

---

```
x = 'cctttffsss'
```

```
output = c2t3f2s3
```

---

input:

```
+-----+-----+
|name    |subjects          |
+-----+-----+
|Alice   |[Math, Physics]   |
|Bob     |[English]          |
|Charlie |[Biology, Chemistry, Math]|
```

Output:

```
+-----+-----+
|name    |subject|
+-----+-----+
|Alice   |Math   |
|Alice   |Physics|
|Bob     |English|
|Charlie |Biology|
|Charlie |Chemistry|
|Charlie |Math   |
+-----+-----+
```

---

## append() vs extend() in Python

### 1 append()

➤ *Adds the entire element as a single item to the list.*

- It does **not** break the element.
- It increases the list length by **1**.

```
lst = [1, 2, 3]
lst.append([4, 5])
print(lst)
```

OUTPUT- [1, 2, 3, [4, 5]]

## Extend()

➤ *Adds each element of the iterable individually to the list.*

- It **iterates** over the argument.
- It increases the list length by **number of items added**.

✓ **Example:**

```
lst = [1, 2, 3]
lst.extend([4, 5])
print(lst)
```

✓ **Output:**

[1, 2, 3, 4, 5]

| Feature               | append()     | extend()                       |
|-----------------------|--------------|--------------------------------|
| Adds element as       | Single item  | Multiple items (iterates)      |
| Input type            | Any object   | Iterable (list, tuple, string) |
| List length increases | +1           | +len(iterable)                 |
| Common use case       | Add one item | Merge two lists                |